

Matn generatsiya va ikki tomonlama RNN

Tabiiy tilni qayta ishlash — 9-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №9

29-iyun 2026

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiya
- 3 Temperature sampling
- 4 Portlovchi gradient va clipping
- 5 Ikki tomonlama (bidirectional) RNN
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L8/P8 da LSTM tasniflashda oxirgi holat h_T dan **bitta** sinf chiqardik (ko'p-dan-bir). Matn **yaratish** uchun chiqish qanday bo'lishi kerak?

2-savol

L7–L8 da **vanishing** gradientni ko'rdik. Uning teskarisi — gradient **portlashi** (exploding) — nima va qanday to'xtatamiz?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L8/P8 da LSTM tasniflashda oxirgi holat h_T dan **bitta** sinf chiqardik (ko'p-dan-bir). Matn **yaratish** uchun chiqish qanday bo'lishi kerak?

1-javob

Har qadamda **keyingi so'z** taqsimoti (ko'p-dan-ko'p). So'z tanlab, uni qaytadan kirita-miz — **autoregressiv** generatsiya.

2-savol

L7–L8 da **vanishing** gradientni ko'rdik. Uning teskarisi — gradient **portlashi** (exploding) — nima va qanday to'xtatamiz?

2-javob

Gradient juda kattalashadi $\Rightarrow$ o'qish beqaror. Yechim: **gradient clipping** (3-tsikl).

Siz bugungi dars oxirida quyidagilarni bajara olasiz:

- ➊ **Tushuntira olasiz:** autoregressiv matn generatsiya arxitekturasini (bashorat → tanlash → qaytadan kiritish).
- ➋ **Hisoblay olasiz:** temperature softmax ni qo'lda — $T=1$: $p(\text{nlp})=0.665$, $T=0.5$: $p(\text{nlp})=0.867$.
- ➌ **Tushuntira olasiz:** portlovchi gradient va gradient clipping.
- ➍ **Tushuntira olasiz:** ikki tomonlama (bidirectional) RNN va uning generatsiyadagi chegarasi.

L6 dan: softmax. L7–L8 dan: RNN/LSTM, vanishing gradient. Matematikadan: $\exp$, norma ($\| \cdot \|$), vektorlarni birlashtirish.

Bugungi dars rejasi:

Tsikl	Mavzu	Vaqt	Hosil natija	bo'luvchi
1	Autoregressiv matn generatsiya	16 daq	keyingi so'z tanlash	
2	Temperature sampling	18 daq	$p(\text{nlp}): 0.665 \rightarrow 0.867$	
3	Portlovchi gradient va clipping	16 daq	norma $5 \rightarrow 2$	
4	Ikki tomonlama (bidirectional) RNN	16 daq	forward + backward	
	Xulosa va 9-amaliyotga ko'rsatmalar	10 daq	Ertaga nima qurishingiz	

Bu dars RNN ni “ijodkor” qiladi

Tasniflashdan (L7–L8) matn **yaratish**ga o'tamiz. Bugungi temperature $T=1$ da $p(\text{nlp})=0.665$ ertangi P9 da assert bo'ladi.

Bugungi dars uchun muammo: bir xil matn qayta-qayta chiqadi

Sodda yechim qayerda qiynaladi:

Ochko'z (greedy) tanlash zerikarli

Har qadam eng ehtimoliy so'zni (argmax) olsak, model **bir xil** naqshni takrorlaydi: “men men men...” — ijodkorlik yo'q.

Faqat o'tmishni ko'rish

Oddiy RNN faqat **chapdan** o'qiydi. Ba'zi vazifalarda (teglash) o'ng kontekst ham kerak.

Bugungi yechimlar:

- 1 **Sampling + temperature**: tasodifiylilikni boshqarib, ijodkorlikni sozlaymiz.
- 2 **Gradient clipping**: portlovchi gradientni jilovlaymiz.
- 3 **Bidirectional RNN**: o'ng va chap kontekstni birlashtiramiz.

Bugun RNN ni **generativ** va **ikki tomonlama** qilamiz.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiya
- 3 Temperature sampling
- 4 Portlovchi gradient va clipping
- 5 Ikki tomonlama (bidirectional) RNN
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: bitta so'z yoz, uni qaytadan o'qi, davom et

Matn generatsiya — **zanjirli** bashorat:

- 1 Joriy holatdan keyingi so'z taqsimotini olamiz.
- 2 Bitta so'zni **tanlaymiz** (argmax yoki sampling).
- 3 Uni **keyingi kirish** qilib qaytadan beramiz.
- 4 Takrorlaymiz (to'xtash belgisigacha).

Tasniflash — ko'p-dan-**bir** ($h_T \rightarrow \text{sinf}$).
Generatsiya — ko'p-dan-**ko'p**: har qadam chiqish bor va u keyingi kirishga aylanadi.

Autoregressiv generatsiya: rasmiy ta'rif

$$P(w_1 \dots w_n) = \prod_{t=1}^n P(w_t \mid w_1 \dots w_{t-1}), \quad P(w_t \mid \cdot) = \text{softmax}(W_o h_t)$$

bunda: h_t — RNN/LSTM yashirin holati (o'tmish xulosasi); W_o — chiqish qatlami; $P(w_t \mid \cdot)$ — lug'at ustidan keyingi so'z taqsimoti; tanlangan w_t keyingi qadam kirishi bo'ladi. Bu — L5 dagi til modeli g'oyasining neyron varianti: kontekst sobit oyna emas, butun o'tmish (h_t orqali).

1-qadam. Tasniflash: faqat oxirgi holat h_T ishlatiladi $\Rightarrow$ bitta chiqish.

Keltirib chiqarish: tasniflashdan generatsiyaga

1-qadam. Tasniflash: faqat oxirgi holat h_T ishlatiladi $\Rightarrow$ bitta chiqish. **2-qadam.** Generatsiya: **har** qadamda chiqish kerak — $P(w_t | w_{<t}) = \text{softmax}(W_o h_t)$.

Keltirib chiqarish: tasniflashdan generatsiyaga

1-qadam. Tasniflash: faqat oxirgi holat h_T ishlatiladi $\Rightarrow$ bitta chiqish. **2-qadam.** Generatsiya: **har** qadamda chiqish kerak — $P(w_t | w_{<t}) = \text{softmax}(W_o h_t)$. **3-qadam.** Tanlangan w_t ni $t+1$ qadam **kirishi** qilamiz (autoregressiya). Shu sababli generatsiya ketma-ket (bir vaqtda emas).

Xulosa: bir xil RNN, faqat har qadam chiqishi keyingi kirishga ulanadi — zanjir hosil bo'ladi.

Qo'lda misol: bitta generatsiya qadami (greedy)

Vocab = [men, kitob, non]; logitlar $z = [2.0, 1.0, 0.0]$.

Softmax va tanlov:

$$\begin{aligned} p &= \frac{[e^2, e^1, e^0]}{e^2 + e^1 + e^0} \\ &= \frac{[7.39, 2.72, 1.0]}{11.11} = [0.665, 0.245, 0.090] \end{aligned}$$

argmax $\Rightarrow$ **men** (keyingi kirish).

Greedy har doim men ni oladi $\Rightarrow$ keyingi qadam ham o'xshash $\Rightarrow$ takror. Ijodkorlik uchun **sampling** kerak (2-tsikl).

Sizning vazifangiz: boshqa logitlar uchun greedy tanlov

Savol — 60 soniya

Vocab = [men, kitob, non], logitlar $z = [0.0, 1.0, 2.0]$:

greedy (argmax) qaysi so'zni tanlaydi?

Sizning vazifangiz: boshqa logitlar uchun greedy tanlov

Savol — 60 soniya

Vocab = [men, kitob, non], logitlar $z = [0.0, 1.0, 2.0]$:

greedy (argmax) qaysi so'zni tanlaydi?

Javob

Eng katta logit — $z_3 = 2.0$ (non). $\Rightarrow \text{argmax} = \text{non}$.

Softmax tartibni o'zgartirmaydi (monoton), shuning uchun greedy uchun shunchaki eng katta logitni olish kifoya.

Kod $\leftrightarrow$ formula: autoregressiv generatsiya

```
1 import numpy as np
2
3 def generate(model, h, start, n):
4     seq = [start]
5     x = start
6     for _ in range(n):
7         logits, h = model.step(h, x)    #  $P(w|.)$ 
8         p = np.exp(logits) / np.exp(logits).sum()
9         x = int(np.argmax(p))           # greedy
10        seq.append(x)
11    return seq
```

Kod $\rightarrow$ formula:

Kod	Formula
model.step	$h_t, W_o h_t$
exp/sum	softmax
argmax(p)	w_t tanlov
x = ...	qaytadan kirish

P9 da m09 TextGenerator aynan shu zanjirni char-darajasida quradi.

Generatsiya tuzog'i: greedy takrorlanishga olib keladi

Takrorlanish halqasi

Greedy har doim eng ehtimoliy so'zni oladi $\Rightarrow$ model tez-tez **bir xil iboraga** "tushib qoladi" va uni qaytaraveradi.

Yechim: taqsimotdan **tasodifiy** tanlash (sampling) va **temperature** bilan tasodifiylik darajasini sozlash — 2-tsikl.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiya
- 3 Temperature sampling**
- 4 Portlovchi gradient va clipping
- 5 Ikki tomonlama (bidirectional) RNN
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: temperature — “ijodkorlik regulyatori”

Logitlarni tanlashdan oldin T ga **bo'lamiz**:

- **Past** $T (< 1)$: taqsimot “o'tkirlashadi” — ishonchli, deterministikroq.
- **Yuqori** $T (> 1)$: taqsimot tekislanadi — tasodifiy, ijodkorroq.
- $T=1$: oddiy softmax.

$T \rightarrow 0$: argmax (greedy). $T \rightarrow \infty$: bir tekis (tasodifiy). T — ijodkorlik va ishonchlilik orasidagi murosa.

Temperature softmax: rasmiy ta'rif

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

bunda: z_i — i -so'z logiti; $T > 0$ — temperatura; $T=1$ oddiy softmax; $T < 1$ o'tkirlashtiradi (eng katta logit ustun); $T > 1$ tekislaydi. **Chegaralar:**

$$T \rightarrow 0^+ : p \rightarrow \operatorname{argmax} \text{ (one-hot)}; \quad T \rightarrow \infty : p \rightarrow \text{bir tekis}$$

Keltirib chiqarish: nega T ga bo'lish ishlaydi?

1-qadam. Softmax logitlar **farqi**ga sezgir: $p_i \propto e^{z_i}$.

Keltirib chiqarish: nega T ga bo'lish ishlaydi?

1-qadam. Softmax logitlar **farqiga** sezgir: $p_i \propto e^{z_i}$. **2-qadam.** Logitlarni T ga bo'lsak, farqlar T marta o'zgaradi: $z_i/T - z_j/T = (z_i - z_j)/T$.

Keltirib chiqarish: nega T ga bo'lish ishlaydi?

1-qadam. Softmax logitlar **farqiga** sezgir: $p_i \propto e^{z_i}$. **2-qadam.** Logitlarni T ga bo'lsak, farqlar T marta o'zgaradi: $z_i/T - z_j/T = (z_i - z_j)/T$. **3-qadam.** $T < 1$ farqlarni **kattalashtiradi** (o'tkir taqsimot); $T > 1$ **kichraytiradi** (tekis taqsimot):

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Xulosa: T — logit farqlarining “kuchaytirgichi”. Bitta parametr bilan ijodkorlikni sozlaymiz.

Qo'lda hisob: temperature — $p(\text{nlp}): 0.665 \rightarrow 0.867$

$$e^3 \approx 20.09, \quad e^1 \approx 2.72, \quad e^2 \approx 7.39.$$

$T = 1$ (**oddiy softmax**):

$$\begin{aligned} p(\text{nlp}) &= \frac{e^3}{e^3 + e^1 + e^2} \\ &= \frac{20.09}{30.19} \approx \mathbf{0.665} \end{aligned}$$

Past T (0.5) eng ehtimoliy so'z (nlp) ustunligini **kuchaytirdi**:
 $0.665 \rightarrow 0.867$.

$T = 0.5$ ($z/T = [6, 2, 4]$):

$$\begin{aligned} p(\text{nlp}) &= \frac{e^6}{e^6 + e^2 + e^4} \\ &= \frac{403.4}{465.4} \approx \mathbf{0.867} \end{aligned}$$

Ertangi P9 birinchi assert:
`abs(p_nlp_T1 - 0.665) < 1e-3`
Ma'ruza L9 [12]-slyd

Sizning vazifangiz: $T = 0.5$ da $p(\text{qiziq})$

Savol — 90 soniya

$T = 0.5$, $z/T = [6, 2, 4]$, $e^6 \approx 403.4$, $e^2 \approx 7.39$, $e^4 \approx 54.60$ (yig'indi ≈ 465.4):
 $p(\text{qiziq}) = ?$

Sizning vazifangiz: $T = 0.5$ da $p(\text{qiziq})$

Savol — 90 soniya

$T = 0.5$, $z/T = [6, 2, 4]$, $e^6 \approx 403.4$, $e^2 \approx 7.39$, $e^4 \approx 54.60$ (yig'indi ≈ 465.4):
 $p(\text{qiziq}) = ?$

Javob

$$p(\text{qiziq}) = \frac{e^4}{465.4} = \frac{54.60}{465.4} \approx \mathbf{0.117}$$

$T=1$ da 0.245 edi $\rightarrow T=0.5$ da 0.117 ga **pasaydi**. Past temperatura yetakchi bo'lmagan so'zlarni siqib chiqaradi.

Kod $\leftrightarrow$ formula: temperature softmax

```
1 import numpy as np
2
3 def temp_softmax(z, T=1.0):
4     z = np.asarray(z) / T           # logitlarni
5                                     # T ga bo'lish
6     e = np.exp(z - z.max())         # barqaror
7                                     # softmax
8     return e / e.sum()
9
10 z = [3.0, 1.0, 2.0]
11 print(temp_softmax(z, 1.0)[0])    # 0.665
12 print(temp_softmax(z, 0.5)[0])    # 0.867
```

Kod $\rightarrow$ formula:

Kod	Formula
z / T	z_i / T
$\exp(z - \max)$	barqaror $e^{z_i / T}$
$e / e.\text{sum}()$	p_i

$z - z.\text{max}()$ — overflow oldini olish;
natijaga ta'sir qilmaydi. P9 da m09
shu funktsiyani ishlatadi.

Temperature tuzog'i: chetki qiymatlar yomon

Ikki chekka

- Juda past T : takrorlanish, zerikarli matn (greedy kabi).
- Juda yuqori T : ma'nosiz, tasodifiy "shovqin".
- $T=0$: bo'linish xatosi (kodda alohida ishlov).

Amalda $T \in [0.7, 1.0]$ ko'pincha yaxshi muvozanat. Qo'shimcha: top- k yoki top- p (nucleus) sampling.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiya
- 3 Temperature sampling
- 4 Portlovchi gradient va clipping
- 5 Ikki tomonlama (bidirectional) RNN
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: gradient ham portlashi mumkin

L7–L8 da gradient **so'ndi** (vanishing). Teskari muammo ham bor: agar bo'g'inlar > 1 bo'lsa, ko'paytma **eksponensial o'sadi** — gradient **portlaydi**.

Natija: og'irliklar sakrab o'zgaradi, loss “NaN” ga ketadi — o'qish buziladi.

Yechim oson: gradient normasi belgilangan chegaradan oshsa, uni **qayta masshtablaymiz** (clip) — yo'nalish saqlanadi, kattalik cheklanadi.

Gradient clipping: rasmiy ta'rif

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \|\mathbf{g}\| \leq \tau \\ \frac{\tau}{\|\mathbf{g}\|} \mathbf{g} & \|\mathbf{g}\| > \tau \end{cases}$$

bunda: $\mathbf{g}$ — gradient vektori; $\|\mathbf{g}\|$ — uning normasi (uzunligi); τ — chegara (threshold). Norma τ dan oshsa, vektor **yo'nalishi saqlanib**, uzunligi τ ga keltiriladi. Faqat **kattalik** o'zgaradi, yo'nalish emas — shuning uchun o'qish yo'nalishi to'g'ri qoladi.

Keltirib chiqarish: clip nega yo'nalishni saqlaydi?

1-qadam. Gradient $\mathbf{g}$ — yo'nalish $\times$ kattalik: $\mathbf{g} = \|\mathbf{g}\| \cdot \hat{\mathbf{g}}$, $\hat{\mathbf{g}}$ birlik vektor.

Keltirib chiqarish: clip nega yo'nalishni saqlaydi?

1-qadam. Gradient $\mathbf{g}$ — yo'nalish $\times$ kattalik: $\mathbf{g} = \|\mathbf{g}\| \cdot \hat{\mathbf{g}}$, $\hat{\mathbf{g}}$ birlik vektor. **2-qadam.** Clip kattalikni τ ga almashtiradi, $\hat{\mathbf{g}}$ ni o'zgartirmaydi:

$$\frac{\tau}{\|\mathbf{g}\|} \mathbf{g} = \frac{\tau}{\|\mathbf{g}\|} \cdot \|\mathbf{g}\| \hat{\mathbf{g}} = \tau \hat{\mathbf{g}}$$

Keltirib chiqarish: clip nega yo'nalishni saqlaydi?

1-qadam. Gradient $\mathbf{g}$ — yo'nalish $\times$ kattalik: $\mathbf{g} = \|\mathbf{g}\| \cdot \hat{\mathbf{g}}$, $\hat{\mathbf{g}}$ birlik vektor. **2-qadam.** Clip kattalikni τ ga almashtiradi, $\hat{\mathbf{g}}$ ni o'zgartirmaydi:

$$\frac{\tau}{\|\mathbf{g}\|} \mathbf{g} = \frac{\tau}{\|\mathbf{g}\|} \cdot \|\mathbf{g}\| \hat{\mathbf{g}} = \tau \hat{\mathbf{g}}$$

3-qadam. Demak natijaning normasi aynan τ , yo'nalishi $\hat{\mathbf{g}}$ — o'zgarmagan.

Xulosa: portlovchi gradient jilovlanadi, optimizatsiya yo'nalishi buzilmaydi.

Qo'lda misol: gradientni clip qilamiz

$$\|\mathbf{g}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5 > 2 \Rightarrow \text{clip kerak.}$$

Qayta masshtablaymiz:

$$\begin{aligned}\mathbf{g} &\leftarrow \frac{\tau}{\|\mathbf{g}\|} \mathbf{g} = \frac{2}{5} [3, 4] \\ &= [1.2, 1.6]\end{aligned}$$

Yo'nalish $[3, 4]$ saqlandi (ikkalasi 0.4 ga ko'paytirildi), uzunlik $5 \rightarrow 2$. Portlash jilovlandi.

$$\text{Tekshiruv: } \|[1.2, 1.6]\| = \sqrt{1.44 + 2.56} = \sqrt{4} = 2.$$

Sizning vazifangiz: yana bir clip

Savol — 90 soniya

Gradient $\mathbf{g} = [6, 8]$, chegara $\tau = 5$:

$\|\mathbf{g}\| = ?$ **va clip qilingan $\mathbf{g} = ?$**

Sizning vazifangiz: yana bir clip

Savol — 90 soniya

Gradient $\mathbf{g} = [6, 8]$, chegara $\tau = 5$:

$\|\mathbf{g}\| = ?$ **va clip qilingan $\mathbf{g} = ?$**

Javob

$$\|\mathbf{g}\| = \sqrt{36 + 64} = \sqrt{100} = 10 > 5 \Rightarrow \mathbf{g} \leftarrow \frac{5}{10} [6, 8] = [3, 4]$$

Tekshiruv: $\|[3, 4]\| = 5 = \tau$. Yo'nalish o'zgarmadi (yarmiga kichraydi), norma $10 \rightarrow 5$.

Kod $\leftrightarrow$ formula: gradient clipping

```
1 import numpy as np
2
3 def clip_grad(g, tau):
4     norm = np.linalg.norm(g)
5     if norm > tau:
6         g = g * (tau / norm)      # qayta
                                   # masshtab
7     return g
8
9 print(clip_grad(np.array([3.0, 4.0]), 2))  #
    [1.2 1.6]
10
11 # PyTorch:
12 # torch.nn.utils.clip_grad_norm_(model.
    parameters(), tau)
```

Kod $\rightarrow$ formula:

Kod	Formula
norm(g)	$\ g\ $
tau/norm	$\tau / \ g\ $
g * ...	$\tau \hat{g}$

Amalda clip_grad_norm_backward dan keyin, optimizer qadamidan oldin chaqiriladi.

Clipping tuzog'i: sabab emas, alomatni davolaydi

Cheklov

Clipping portlashni **to'xtatadi**, ammo uning sababini (masalan yomon initsializatsiya, juda katta η) hal qilmaydi. Faqat clipping'ga tayanish — yetarli emas.

Yondashuv: clipping + to'g'ri η + LSTM/GRU (vanishing uchun) + yaxshi init. Hammasi birga barqaror o'qishni beradi.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiya
- 3 Temperature sampling
- 4 Portlovchi gradient va clipping
- 5 Ikki tomonlama (bidirectional) RNN
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: ikki marta o'qiybiz — oldinga va orqaga

Ba'zi so'zning ma'nosi **keyingi** so'zlarga bog'liq:
“*u ____ ni ko'rdi*” — bo'shliqdagi so'z o'ngdagi kontekstga qarab aniqlanadi.

Bidirectional RNN: bitta RNN chapdan o'ngga, ikkinchisi o'ngdan chapga o'qiydi; ikkala holatni **birlashtiramiz**.

Har token uchun **to'liq** kontekst (chap + o'ng) hosil bo'ladi — teglash, NER, tasniflashda foydali.

Bidirectional RNN: rasmiy ta'rif

$$\vec{h}_t = \text{RNN}_f(x_{1:t}), \quad \overleftarrow{h}_t = \text{RNN}_b(x_{t:T}), \quad h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

bunda: $\vec{h}_t$ — oldinga (chapdan) holat; $\overleftarrow{h}_t$ — orqaga (o'ngdan) holat; $[\ ;]$ — birlashtirish; h_t — t token uchun to'liq (ikki tomonlama) ifoda, o'lchami $2 \times$. Diqqat: $\overleftarrow{h}_t$ **butun** ketma-ketlikni talab qiladi — shuning uchun bidirectional **generatsiyada ishlamaydi** (kelajak noma'lum).

Keltirib chiqarish: nega ikki holatni birlashtiramiz?

1-qadam. Oldinga RNN $\vec{h}_t$ faqat $x_1 \dots x_t$ ni biladi (chap kontekst).

Keltirib chiqarish: nega ikki holatni birlashtiramiz?

1-qadam. Oldinga RNN $\vec{h}_t$ faqat $x_1 \dots x_t$ ni biladi (chap kontekst). **2-qadam.** Orqaga RNN $\overleftarrow{h}_t$ faqat $x_t \dots x_T$ ni biladi (o'ng kontekst).

Keltirib chiqarish: nega ikki holatni birlashtiramiz?

1-qadam. Oldinga RNN $\vec{h}_t$ faqat $x_1 \dots x_t$ ni biladi (chap kontekst). **2-qadam.** Orqaga RNN $\overleftarrow{h}_t$ faqat $x_t \dots x_T$ ni biladi (o'ng kontekst). **3-qadam.** Birlashtirsak, $h_t = [\vec{h}_t; \overleftarrow{h}_t]$ — **ikkala** tomon. Keyingi qatlam to'liq kontekstdan foydalanadi.

Xulosa: ikki tomonlama o'qish teglash/tasniflash aniqligini oshiradi — ammo butun ketma-ketlik oldindan kerak.

Qo'lda misol: ikki tomonlama holatni birlashtiramiz

$$\vec{h}_t = [0.5, 0.2] \text{ (oldinga)}, \quad \overleftarrow{h}_t = [0.1, 0.7] \text{ (orqaga)}.$$

Birlashtiramiz:

$$\begin{aligned} h_t &= [\vec{h}_t; \overleftarrow{h}_t] \\ &= [0.5, 0.2, 0.1, 0.7] \end{aligned}$$

Endi h_t ham chap ($[0.5, 0.2]$), ham o'ng ($[0.1, 0.7]$) kontekstni o'z ichiga oladi — keyingi qatlam ikkalasini ko'radi.

O'lcham $2 \rightarrow 4$ (ikki baravar).

Sizning vazifangiz: boshqa token uchun birlashtirish

Savol — 60 soniya

$$\vec{h}_t = [0.9, 0.0], \quad \overleftarrow{h}_t = [0.3, 0.4]:$$

$h_t = ?$ **va uning o'lchami nechta?**

Sizning vazifangiz: boshqa token uchun birlashtirish

Savol — 60 soniya

$$\vec{h}_t = [0.9, 0.0], \quad \overleftarrow{h}_t = [0.3, 0.4]:$$

$h_t = ?$ **va uning o'lchami nechta?**

Javob

$$h_t = [0.9, 0.0, 0.3, 0.4], \quad \text{o'lcham} = 4.$$

Birlashtirish shunchaki ikki vektorni ulaydi — chap holat oldinda, o'ng holat orqada.

Kod $\leftrightarrow$ formula: bidirectional LSTM

```
1 import torch.nn as nn
2
3 bilstm = nn.LSTM(input_size=100, hidden_size=64,
4                 batch_first=True,
5                 bidirectional=True)    # ikki
5                                     tomonlama
6
7 # chiqish o'lchami = 2 * hidden_size = 128
8 # (forward h + backward h birlashtiriladi)
9 out, (h, c) = bilstm(x)    # out: (B, T, 128)
```

Kod $\rightarrow$ formula:

Kod	Formula
bidirectional	$\vec{h}, \overleftarrow{h}$
2*hidden	[;] o'lcham
out	h_t (har token)

Bidirectional — teglash/NER (L10) uchun; **generatsiyada ishlaymaydi** (kelajak kerak).

Bidirectional tuzog'i: generatsiyada ishlatib bo'lmaydi

Sababiyat (causality) buziladi

Orqaga RNN **kelajak** tokenlarni ko'radi. Matn generatsiyada kelajak hali yo'q $\Rightarrow$ bidirectional ishlamaydi.

Qoida: bidirectional — *tushunish* (tasniflash, teglash, NER) uchun; generatsiya esa bir tomonlama (chapdan o'ngga) bo'lishi shart.

Prezentatsiya rejasi

- 1 Kirish: RNN bilan matn yaratamiz
- 2 Autoregressiv matn generatsiya
- 3 Temperature sampling
- 4 Portlovchi gradient va clipping
- 5 Ikki tomonlama (bidirectional) RNN
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: badiiy generatsiya va SOV uchun ikki tomonlama o'qish

1. Badiiy korpus generatsiyasi:

Cho'lpon yoki Hamza asarlarida o'qitilgan LSTM:

- $T=0.5$: o'zbek iboralari naqshini ko'proq **takrorlaydi**.
- $T=1.0$: yangiroq, ammo ba'zan ma'nosiz birikmalar.

2. SOV va bidirectional:

"men kitobni ___ berdim" — kesim oxirida.

Foyda

Bidirectional orqaga o'qib, gap oxiridagi kesimni **oldinroq** tokenlarga "yetkazadi" — teglash/NER uchun ayniqsa muhim.

Sintez: tanlash strategiyasi va yo'nalish

Tanlov	Xulq	Qachon
Greedy (argmax)	deterministik, takrorlanuvchi	aniq javob kerak
Past T (0.5) Yuqori T (1.2)	ishonchli, naqsh takrori ijodiy, xavfli	rasmiy/aniq matn she'r, ijodiy matn
Bir tomonlama Bidirectional	faqat chap kontekst chap + o'ng	generatsiya teglash/tasniflash

Asosiy g'oya

Generatsiya = autoregressiv (bir tomonlama) + temperature bilan boshqariladigan tasodifiylik; tushunish = bidirectional to'liq kontekst. Portlovchi gradient clipping bilan jilovlanadi.

Seminal manba: Cho va boshqalar (2014)

Cho, K., van Merriënboer, B., va boshq. (2014). **Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.** *EMNLP 2014*.

Ahamiyati:

- Encoder–Decoder (seq2seq) g'oyasini va GRU ni kiritdi.
- Generativ ketma-ketlik modellashtirishning asosi — tarjima, xulosalash, suhbat.
- Keyingi mavzular (attention, Transformer) shu poydevorga quriladi.

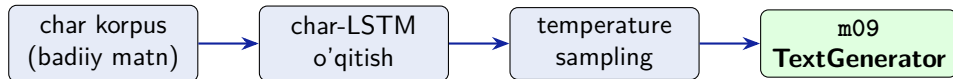
Muhokama savoli (P9 gacha o'ylang)

O'zbek badiiy matnini generatsiya qilishda qaysi temperature (0.5, 0.8, 1.2) eng tabiiy natija beradi deb o'ylaysiz? Nega? (P9 da sinab ko'ramiz.)

Bugungi maqsadlar bajarildi

- ① ✓ **Tushuntirdik:** autoregressiv generatsiya — bashorat $\rightarrow$ tanlash $\rightarrow$ qaytadan kiritish.
- ② ✓ **Hisobladik:** temperature softmax — $T=1$: $p(\text{nlp})=0.665$, $T=0.5$: 0.867.
- ③ ✓ **Tushuntirdik:** portlovchi gradient va clipping (norma $5 \rightarrow 2$).
- ④ ✓ **Tushuntirdik:** bidirectional RNN ($h_t = [\vec{h}; \overleftarrow{h}]$) va uning generatsiyadagi chegarasi.

Ertaga P9: char-darajali LSTM bilan matn generatsiya qilamiz



P9 da qurasiz:

- m09 TextGenerator: char-LSTM train/generate
- Temperature bilan generatsiya (0.3/0.7/1.2)
- Temperature softmax $p(\text{nlp})$ ni tekshirish

Birinchi assert (bu slayd [I2]):

```
assert abs(p_nlp_T1  
- 0.665) < 1e-3
```

*# Ma'ruza L9 [I2]-slayd bilan
solishtiring (temperature)*

- ① Cho, K., van Merriënboer, B., va boshq. (2014). *Learning Phrase Representations using RNN Encoder–Decoder...* EMNLP 2014.
- ② Graves, A. (2013). *Generating Sequences With Recurrent Neural Networks*. arXiv:1308.0850. (Matn generatsiya, sampling.)
- ③ Pascanu, R., Mikolov, T., Bengio, Y. (2013). *On the difficulty of training RNNs*. ICML. (Gradient clipping.)
- ④ Schuster, M., Paliwal, K. (1997). *Bidirectional Recurrent Neural Networks*. IEEE Trans. Signal Processing.
- ⑤ `torch.nn.LSTM(bidirectional=True)` / `clip_grad_norm_` hujjatlari.

Ilova S1: top- k va top- p (nucleus) sampling

Temperature — yagona yo'l emas. Boshqa keng tarqalgan strategiyalar:

- **top- k** : faqat eng yuqori k ta so'z orasidan tanlaymiz (qolganlari 0).
- **top- p (nucleus)**: yig'indi ehtimoli p ga yetguncha eng yuqori so'zlarni olamiz.

bunda: ikkalasi ham “dumi uzun” kam ehtimolli so'zlarni kesib, ma'nosiz tanlovlarni kamaytiradi; temperature bilan birga ishlatiladi.

Ilova S2: temperature chegaralarini keltirib chiqarish

$T \rightarrow 0^+$: eng katta z_i uchun $e^{z_i/T}$ boshqalardan beqiyos katta bo'ladi $\Rightarrow p \rightarrow \text{one-hot (argmax)}$.

$$\lim_{T \rightarrow 0^+} \frac{e^{z_i/T}}{\sum_j e^{z_j/T}} = \begin{cases} 1 & z_i = \max_j z_j \\ 0 & \text{aks holda} \end{cases}$$

$T \rightarrow \infty$: $z_i/T \rightarrow 0$ barcha i uchun $\Rightarrow e^0 = 1 \Rightarrow$ bir tekis $p_i = 1/|V|$. **bunda:** shuning uchun T greedy ($T=0$) va tasodifiy ($T=\infty$) orasidagi silliq o'tish.